Spring Auditing

- What is Auditing?
- Auditing Levels Overview
  - Level 1 – Technical Audit (Metadata)
  - Level 2 – Historical Audit (Entity History)
- Spring Data JPA Auditing
  - What Spring Data Auditing Tracks
  - How Spring Data Auditing Works
  - Configuration
    - Enable JPA Auditing
    - Base Auditable Entity
    - Why Use @MappedSuperclass?
    - AuditableEntity Implementation
    - Why @EntityListeners(AuditingEntityListener.class)?
  - What Spring Data Auditing Does NOT Do
  - When Level 1 Auditing Is ENOUGH
- Hibernate Envers (Optional / Advanced)
  - What Envers Does
  - Revision Concept
  - What @Audited actually does
- Business Auditing (Action / Trace Audit)
  - Custom Audit Annotation
  - Audit Aspect (AOP)
  - Purpose of Action Auditing

What is Auditing?

Auditing is the ability to trace:

- **WHO** performed an action
- **WHAT** was changed
- **WHEN** it happened
- **(Sometimes) WHY** it happened

Auditing is essential in enterprise applications to ensure:

- Traceability
- Accountability
- Debugging

- Compliance (banking, finance, regulated systems)

Auditing = the ability to trace WHO did WHAT, WHEN, and SOMETIMES WHY on business data, with history.

There are **2 levels**:

1. **Technical audit** (timestamps, user)
2. **Business audit / history** (old values, versions)

Auditing Levels Overview

There are **two main levels of auditing** commonly used in Spring applications:

Level 1 – Technical Audit (Metadata)

Tracks **who** created or modified data and **when**.

- Lightweight.
- Built into Spring Data JPA.
- No historical versions

Level 2 – Historical Audit (Entity History)

Tracks **old values**, **versions**, and **full change history**.

- Full traceability
- More complex
- Larger database footprint

Spring Data JPA Auditing

This section describes how Level 1 auditing is implemented using Spring Data JPA.

What Spring Data Auditing Tracks

Spring Data JPA Auditing automatically manages the following fields:

- `createdAt`
- `updatedAt`
- `createdBy`
- `lastModifiedBy`

This information is stored **on the entity itself**.

How Spring Data Auditing Works

Spring uses **JPA lifecycle events** (`persist` , `update`) to inject audit values automatically.

This is done via:

- An **entity listener**
- A **current user provider** (`AuditorAware`)

Configuration

Enable JPA Auditing

```java
@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorAware")
public class JpaAuditingConfig {

    // AuditorAware = Spring hook to tell JPA who is the current user
    @Bean
    public AuditorAware<String> auditorAware() {
        return () -> {
            var auth = SecurityContextHolder.getContext().getAuthentication();
            if (auth == null || !auth.isAuthenticated()) {
                return Optional.empty();
            }
            return Optional.of(auth.getName());
        };
    }
}

```

Role of `AuditorAware` :

- Called by Spring when an entity is persisted or updated
- Returns the current authenticated user

Base Auditable Entity

Why Use `@MappedSuperclass` ?

- This class is **NOT an entity**
- No database table is created
- Its fields are inherited by all child entities
- Prevents duplication of audit fields

AuditableEntity Implementation

```java
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Getter
@Setter
```

```java
@NoArgsConstructor
@AllArgsConstructor
@SuperBuilder
public abstract class AuditableEntity {

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;

    @CreatedDate
    @Column(updatable = false, nullable = false)
    private Instant createdAt;

    @LastModifiedBy
    private String lastModifiedBy;

    @LastModifiedDate
    private Instant updatedAt;
}
```

Why `@EntityListeners(AuditingEntityListener.class)`?

This annotation:

- Registers a JPA lifecycle listener
- Intercepts `persist` and `update` events
- Automatically fills audit fields

Without it:

- `@CreatedDate`, `@CreatedBy`, etc. are ignored

What Spring Data Auditing Does NOT Do

Spring Data Auditing **does NOT**:

- Store old values
- Track history
- Allow time-travel queries

Example limitation:

> "What was the credit limit last month?"
>
> Cannot be answered with Level 1 auditing.

When Level 1 Auditing Is ENOUGH

Spring Data Auditing is sufficient when:

- You only need current state
- You only care about who created / modified data

- Historical versions are not required
- Business traceability is handled via logs or events

Most applications only need Level 1 auditing.

Hibernate Envers (Optional / Advanced)

Hibernate Envers provides **full entity history**.(technical auditing)

What Envers Does

When an entity is annotated with:

```
@Audited
@Entity
public class CreditLine { ... }

```

Hibernate automatically creates:

- `credit_lines` – current state
- `credit_lines_AUD` – historical versions
- `revinfo` – revision metadata

Each transaction:

- Creates one revision
- Stores old values

Revision Concept

A **revision** represents:

> One transactional change set

Key rule:

> **One transaction = one revision**

What `@Audited` actually does

This line:

```
@Audited
@Entity
public class CreditLine extends TenantEntity

```

means:

- Hibernate Envers is enabled **for this entity**
- Hibernate will automatically:
  - create `credit_lines_AUD`
  - create/use `revinfo`
  - store a new row on **every INSERT / UPDATE / DELETE**
  - store **old values**

Even if you:

- never queried `_AUD` tables
- never customized revisions
- never exposed audit APIs

Envers was still running.

This is not optional behavior.

Business Auditing (Action / Trace Audit)

In many systems, **auditing behavior is more important than auditing data**.

Example:

- Who approved a credit?
- Who triggered a business action?
- How long did it take?
- Did it succeed or fail?

Custom Audit Annotation

```java
/**
 * Marks a method for auditing (traceability).
 * Methods annotated with @Audit will be intercepted by AuditAspect.
 */
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Audit {
    String action() default "";
    String resource() default "";
}
```

Audit Aspect (AOP)

```
```

```java
@Slf4j
@Aspect
@Component
public class AuditAspect {

    @Around("@annotation(audit)")
    public Object auditAction(ProceedingJoinPoint joinPoint, Audit audit) throws Throwable {
        long start = System.currentTimeMillis();
        String actor = SecurityContextHolder.getContext().getAuthentication() != null
                ? SecurityContextHolder.getContext().getAuthentication().getName()
                : "anonymous";

        try {
            Object result = joinPoint.proceed();
            long duration = System.currentTimeMillis() - start;
            log.info("{{\"action\":\"{}\",\"actor\":\"{}\",\"resource\":\"{}\",\"status\":\"SUCCESS\",\"durationMs\":{}}}",
                    audit.action(), actor, audit.resource(), duration);
            String traceId = MDC.get("traceId");

            log.info(
                    "{{\"traceId\":\"{}\",\"actor\":\"{}\",\"action\":\"{}\",\"resource\":\"{}\",\"status\":\"SUCCESS\",\"durationMs\":{}}}",
                    traceId, actor, audit.action(), audit.resource(), duration
            );


            return result;
        } catch (Exception ex) {
            long duration = System.currentTimeMillis() - start;
            log.error("{{\"action\":\"{}\",\"actor\":\"{}\",\"resource\":\"{}\",\"status\":\"ERROR\",\"durationMs\":{},\"error\":\"{}\"}}",
                    audit.action(), actor, audit.resource(), duration, ex.getMessage());
            throw ex;
        }
    }
}

```

Purpose of Action Auditing

This form of auditing tracks:

- Business actions
- User behavior
- Execution time
- Errors
- Trace correlation (via MDC)

This is **business audit**, not data history.

- **Spring Data JPA Auditing** handles technical metadata
- It is simple, lightweight, and sufficient for most applications
- **Business auditing** is often better handled via logs and AOP
- **Hibernate Envers** is optional and should be added only when full history is required
- Auditing should always match real business needs, not be over-engineered